

INSTITUTO NACIONAL DE CIUDAD BARRIOS (INCB)

CÓDIGO 12793

GUÍA DE APRENDIZAJE DE DESARROLLO DE SOFTWARE

CONSULTAS SQL CON FUNCIONES DE AGREGACIÓN Y AGRUPAMIENTO

ESTUDIANTE

EDWIN GEOVANI SOSA SOSA

DOCENTE TITULAR

Prof. Allan René Romero Granados

MÓDULO FORMATIVO

BTVDS 3.8: Desarrollo de Sistemas de Información Gerencial (SIG)

NIVEL Y SECCIÓN

3° Año "A"

FECHA

Jueves 01 de octubre de 2026

1. LA MISIÓN DEL DÍA

En esta guía trabajé con consultas SQL para sacar información de las ventas y de los productos. Primero revisé las tablas que iba a necesitar y después fui haciendo las consultas utilizando las funciones que pedía la actividad.

Utilicé SUM para sumar cantidades y valores, COUNT para contar registros, AVG para sacar promedios y MAX y MIN para encontrar valores mayores y menores. También utilicé GROUP BY para juntar los datos por categoría o empleado y HAVING para poner condiciones a esos grupos.

Con esto pude practicar cómo sacar información de varios registros de una manera más rápida y ordenada.

2. LIENZO DE TRABAJO: MATRIZ DE CONSULTAS GERENCIALES

1. ¿Cuánto dinero total ha ingresado por ventas?

Tablas involucradas: detalle_ventas, productos.

Función utilizada: SUM.

Resultado esperado: Obtener el total de dinero de las ventas.

Para esta consulta utilicé SUM para sumar los valores y obtener el total de las ventas realizadas.

2. ¿Cuáles son las ventas por categoría?

Tablas involucradas: categorias, productos, detalle_ventas.

Función utilizada: SUM y GROUP BY.

Resultado esperado: Ver cuánto se ha vendido en cada categoría.

Aquí agrupé los productos por categoría y después sumé los valores correspondientes a las ventas.

3. ¿Quién es el vendedor con más ventas?

Tablas involucradas: empleados, ventas.

Función utilizada: COUNT, SUM y GROUP BY.

Resultado esperado: Saber cuántas ventas realizó cada empleado y cuánto vendió.

Para hacerlo agrupé las ventas por empleado. Con COUNT conté las ventas y con SUM obtuve el total vendido.

4. ¿Qué categorías superaron los \$1,000?

Tablas involucradas: categorias, productos, detalle_ventas.

Función utilizada: SUM, GROUP BY y HAVING.

Resultado esperado: Mostrar las categorías que tuvieron más de \$1,000 en ventas.

Primero agrupé las ventas por categoría y después utilicé HAVING para dejar solamente las categorías que pasaron de \$1,000.

5. ¿Cuál es el precio promedio de los productos?

Tabla involucrada: productos.

Función utilizada: AVG.

Resultado esperado: Obtener el promedio de los precios.

Utilicé AVG para calcular el precio promedio de los productos registrados.

6. ¿Cuál es el precio más alto de los productos?

Tabla involucrada: productos.

Función utilizada: MAX.

Resultado esperado: Encontrar el precio más alto.

Utilicé MAX para encontrar el valor más alto entre los precios.

7. ¿Cuál es el precio más bajo de los productos?

Tabla involucrada: productos.

Función utilizada: MIN.

Resultado esperado: Encontrar el precio más bajo.

Utilicé MIN para encontrar el precio menor de los productos.

8. ¿Cuántas unidades se han vendido por producto?

Tablas involucradas: productos, detalle_ventas.

Función utilizada: SUM y GROUP BY.

Resultado esperado: Saber cuántas unidades se han vendido de cada producto.

Para esto agrupé los registros por producto y sumé las cantidades vendidas.

3. ANDAMIAJE: CLÁUSULAS SQL Y SU SECUENCIA

Completé las partes que faltaban en la consulta de la siguiente manera:

1. SELECT columnas, SUM(monto) AS total
2. FROM tabla_principal
3. **INNER JOIN** tabla_relacionada ON condicion_enlace
4. WHERE condicion_de_filtrado_inicial
5. **GROUP BY** columna_no_agregada
6. **HAVING** SUM(monto) > valor_minimo
7. ORDER BY total **DESC**

Respuestas

3. **INNER**

5. **GROUP**

6. **HAVING**

7. **DESC**

La consulta completa queda así:

```
SELECT columnas, SUM(monto) AS total
FROM tabla_principal
INNER JOIN tabla_relacionada
ON condicion_enlace
WHERE condicion_de_filtrado_inicial
GROUP BY columna_no_agregada
HAVING SUM(monto) > valor_minimo
ORDER BY total DESC;
```

Al completar el ejercicio pude recordar mejor el orden de las cláusulas. También me quedó más clara la diferencia entre GROUP BY y HAVING.

4. LISTA DE VERIFICACIÓN (CHECKLIST DE ÉXITO)

1. ¿Las 8 consultas ejecutan sin arrojar error ONLY_FULL_GROUP_BY en el gestor?

Sí.

Comprobación: Revisé que las consultas que llevan funciones de agregación estén acompañadas por GROUP BY cuando es necesario.

2. ¿Se aplicaron alias descriptivos con AS?

Sí.

Comprobación: Utilicé AS para poner nombres más fáciles de identificar en los resultados, como total_ventas y ganancia_bruta.

3. ¿La cláusula HAVING se utilizó adecuadamente para filtrar condiciones agregadas?

Sí.

Comprobación: Utilicé HAVING después de GROUP BY para aplicar la condición sobre los resultados agrupados.

4. ¿Los montos calculados tienen coherencia matemática con los datos cargados?

Sí.

Comprobación: Revisé los resultados obtenidos y comprobé que los cálculos tuvieran relación con los datos registrados.

5. ¿El script `consultas_gerenciales.sql` incluye comentarios explicativos en cada consulta?

Sí.

Comprobación: Coloqué comentarios para saber qué hace cada consulta y qué resultado se busca obtener.

5. RUTINA METACOGNITIVA PIN

Positivo

¿Qué consulta SQL compleja te dio mayor satisfacción resolver hoy?

La que más me gustó fue la consulta de las categorías que superaron los \$1,000, porque tuve que usar GROUP BY y HAVING para obtener solamente las categorías que cumplieran con esa condición.

Interesante

¿Por qué es tan valioso poder calcular comisiones o ganancias automáticamente con SQL?

Porque los cálculos se pueden hacer directamente con los datos de la base de datos y no es necesario hacerlos uno por uno. Esto ayuda a ahorrar tiempo y a tener los resultados más rápido.

Necesito Mejorar

¿Qué concepto entre WHERE y HAVING necesitas repasar para no equivocarte?

Necesito practicar más la diferencia entre WHERE y HAVING. Sé que WHERE sirve para filtrar los registros y HAVING para filtrar después de agrupar, pero todavía necesito practicar para no confundirme.